

Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

Übung Nr. 6 Abgabe KW 23

1. Aufgabe

- (a) Implementieren Sie eine Klasse `StringProcessor`, die eine interne Liste mit Strings verwaltet. Nutzen Sie dazu ihre eigene `LinkedList`-Implementierung. Die Klasse soll folgende Methoden bereitstellen:
- **Konstruktor:** Es soll einen Konstruktor geben, der eine leere Liste instanziiert, sowie einen Konstruktor, der eine Liste von Strings als Argument entgegennimmt und die interne Liste mit diesen Werten initialisiert.
 - **add:** Fügt der Liste einen String hinzu.
 - **filter:** Diese Methode gibt eine Liste der Strings zurück, die mit einem an die Methode übergebenen Predicate gefiltert wurden.
 - **applyToAll:** Verändert die Werte der Liste, indem eine an die Methode übergebene Operation auf jeden String der Liste angewendet wird.
 - **mapToInt:** Wandelt alle Strings mithilfe einer übergebenen Funktion in Integer-Werte um und gibt das Ergebnis als neue Liste zurück.
 - **forEach:** Wendet eine Aktion, die als Consumer an die Methode übergeben wird auf alle Strings an, ohne diese zu verändern.
 -
- (b) Testen Sie ihre Implementierung in einer `main`-Methode mit den folgenden Operationen; nutzen Sie dabei Lambda-Ausdrücke und wo möglich Methodenreferenzen.
- Filtern Sie alle Strings, die länger als fünf Zeichen sind.
 - Filtern Sie alle Strings, die mit einem Großbuchstaben beginnen.
 - Wenden Sie auf alle Strings eine Transformation an, die führende und nachfolgende Leerzeichen entfernt.
 - Wenden Sie auf alle Strings eine Transformation an, die den gesamten String in Großbuchstaben umwandelt.
 - Wenden Sie auf alle Strings eine Transformation an, die sie umkehrt (z.B. "Test" → "tseT").
 - Wandeln Sie alle Strings in ihre jeweiligen Längen um.
 - Wandeln Sie alle Strings in Ganzzahlen um, die die Anzahl der Vorkommen des Buchstabens a zählen.
 - Geben Sie alle Strings mit einem vorangestellten "»" aus.

- Geben Sie alle Strings unverändert aus.
- Geben Sie alle Strings mit ihrer jeweiligen Länge in Klammern aus (z. B. "Hallo (5)").

Hinweise

- Verwenden Sie funktionale Interfaces aus `java.util.function`.
- Nutzen Sie ihre eigene `LinkedList`-Implementierung.
- Für einige Aufgaben können Sie die Klassen `Character`¹ und `StringBuilder`² sinnvoll nutzen.

2. Aufgabe (Codeopolis)

Überarbeiten Sie die Klassen `Depot` und `Silo` so, dass Sie die neuen Methoden wie `filter`, `forEach` etc. der Klasse `LinkedList` verwenden, wo immer dies möglich ist. Verwenden Sie dazu Lambda-Ausdrücke. Wo dies möglich ist, geben Sie auch eine alternative Implementierung mit Hilfe einer Methodenreferenz an. Insbesondere in den folgenden Methoden sollten Sie die neuen Methoden verwenden:

- `Depot.Depot(LinkedList<Silo> silos)`
- `Depot.getSilos()`
- `Depot.store(Harvest harvest)`
- `Depot.takeOut(int amount, Game.GrainType grainType)`
- `Depot.takeOut(int amount)`
- `Depot.defragment()`
- `Depot.toString()`
- `Silo.getStockCopy()`
- `Silo.copyStock()`
- `Silo.emptySilo()`
- `Silo.decay()`

Vergleichen Sie die alte und die neue Implementierung und diskutieren Sie die Vor- und Nachteile im Review. Bei Methodenreferenzen sollten Sie erklären können, welche Art von Methodenreferenz Sie verwendet haben.

¹<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html>

²<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/StringBuilder.html>

Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

- ... das für eine Aufgabe passende funktionale Interface auswählen und nutzen können.
- ... in der Lage sein funktionale Interfaces für die Implementierung von Operationen auf einer Datenstruktur zu nutzen.
- ... funktionale Interfaces durch Lambda-Ausdrücke implementieren können.
- ... funktionale Interfaces durch Methodenreferenzen implementieren können.
- ... die verschiedenen Arten von Methodenreferenzen unterscheiden und erklären können.

Zusatzaufgaben

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

1. Aufgabe (Codeopolis)

Implementieren Sie JUnit-Tests für die neuen Funktionen der `LinkedList`.

2. Aufgabe

Schreiben Sie ein Programm, das eine Liste von zufälligen ganzen Zahlen erzeugt, alle ungeraden Zahlen entfernt, die verbleibenden Zahlen sortiert und die sortierte Liste ausgibt. Verwenden Sie dabei Lambda-Ausdrücke. Nutzen Sie die `LinkedList`, die Sie für Codeopolis implementiert haben.

3. Aufgabe

(a) Implementieren Sie mithilfe von Lambda-Ausdrücken folgende Funktionen und nutzen Sie dazu ein geeignetes `Function`-Interface:

- Eine Funktion, die eine Zahl auf ihre Quersumme abbildet.
- Eine Funktion, die eine Zahl auf ihre letzte Ziffer abbildet.
- Eine Funktion, die eine Zahl auf ihre letzten beiden Ziffern abbildet.
- Eine Funktion, die eine Zahl auf ihre letzten drei Ziffern abbildet.

(b) Implementieren Sie die folgenden Prädikate als `Predicate<Integer>` mithilfe von Lambda-Ausdrücken und nutzen Sie dabei wenn notwendig die unter [a](#) implementierten Funktionen:

- `isNegative`: Prüft, ob die Zahl ist kleiner als 0 ist.
- `isZero`: Prüft, ob die Zahl ist gleich 0 ist.
- `isEven`: Prüft, ob die Zahl ist durch 2 teilbar (gerade) ist.
- `isDivisibleBy3`: Prüft, ob die Zahl durch 3 teilbar ist. Eine Zahl ist durch 3 teilbar, wenn die Quersumme durch drei Teilbar ist.
- `isDivisibleBy4`: Prüft, ob die Zahl durch 4 teilbar ist. Eine Zahl ist durch 4 teilbar, wenn die letzten zwei Ziffern durch 4 teilbar sind.
- `isDivisibleBy5`: Prüft, ob die Zahl durch 5 teilbar ist. Eine Zahl ist durch 5 teilbar, wenn die letzte Ziffer 0 oder 5 ist.
- `isDivisibleBy8`: Prüft, ob die Zahl durch 8 teilbar ist. Eine Zahl ist durch 8 teilbar, wenn die letzten drei Ziffern durch 8 teilbar sind.
- `isDivisibleBy9`: Prüft, ob die Zahl durch 9 teilbar ist. Eine Zahl ist durch 9 teilbar, wenn die Quersumme durch 9 teilbar ist.
- `isDivisibleBy10`: Prüft, ob die Zahl durch 10 teilbar ist. Eine Zahl ist durch 10 teilbar, wenn die letzte Ziffer eine 0 ist.
- `isSmaller10000`: Prüft, ob die Zahl kleiner 10000 ist.
- `isMersenne`: Prüft, ob die Zahl eine sogenannte Mersenne-Zahl der Form $2^n - 1$ ist ³.

³ [Wikipedia: Mersenne number](#)

- `isPalindrome`: Prüft, ob die Ziffernfolge der Zahl ein Palindrom ist, d. h. sie liest sich vorwärts und rückwärts gleich. ⁴
 - `isPrime`: Prüft, ob die Zahl eine Primzahl ist.
 - `isFibonacci`: Prüft, ob die Zahl eine Fibonaccizahl ist.
- (c) Implementieren Sie mithilfe von Lambda-Ausdrücken Funktionen, die die folgenden Prädikate erzeugen, und nutzen Sie dazu ein geeignetes `Function`-Interface:
- `isDivisorOf(x)`: Prüft, ob die Zahl ein Teiler von x ist.
 - `isMultipleOf(x)`: Prüft, ob die Zahl ein Vielfaches von x ist.
 - `endsWith(x)`: Prüft, ob die Zahl mit x endet.
 - `isSmaller(x)`: Prüft, ob die Zahl kleiner x ist.
- (d) Kombinieren Sie die Prädikate aus den Aufgabenteil **b** und **c** zu Prädikaten, die folgendes berechnen:
- `isOdd`: Die Zahl ist ungerade.
 - `isPositive`: Die Zahl ist positiv.
 - `isPositiveEven`: Die Zahl ist positiv und gerade.
 - `isDivisibleBy6`: Eine Zahl ist durch 6 teilbar, wenn sie gerade ist und durch 3 teilbar ist.
 - `isDivisibleBy12`: Eine Zahl ist durch 12 teilbar, wenn sie durch 3 und 4 teilbar ist.
 - `isDivisibleBy14`: Eine Zahl ist durch 14 teilbar, wenn sie gerade und durch 7 teilbar ist.
 - `isDivisibleBy15`: Eine Zahl ist durch 15 teilbar, wenn sie durch 3 und durch 5 teilbar ist.
 - `isHarshad`: Die Zahl ist durch ihre eigene Quersumme teilbar ⁵.
 - `hasTwinPrime`: Eine Primzahl p hat einen Primzahlzwilling wenn $p - 2$ oder $p + 2$ auch eine Primzahl ist.
 - `isSophieGermainPrime`: Eine Primzahl p ist eine Sophie Germain Primzahl wenn $2p + 1$ auch eine Primzahl ist.
 - `isNotSquare`: Eine Zahl kann keine Quadratzahl sein, wenn sie auf 2, 3, 7 oder 8 endet.
 - `isSquareCandidate`: Eine Zahl kann eine Quadratzahl sein, wenn sie nicht auf 2, 3, 7 oder 8 endet.
 - `isPrimeFibonacci`: Die Zahl ist eine Primzahl und eine Fibonacci-Zahl.
 - `isPrimeSmaller(x)`: Die Zahl ist eine Primzahl kleiner x .
 - `isOddHarshadSmaller(x)`: Die Zahl ist eine ungerade Harshad-Zahl kleiner x .
- (e) Testen Sie alle Prädikate für die Zahlen kleiner n , wobei n als Programmparameter übergeben werden soll.

4. Aufgabe (Codeopolis)

- (a) Erweitern Sie die Klasse `LinkedList` um eine Methode `sum`. Diese Methode soll eine Funktion entgegennehmen, die ein Element der Liste auf einen Double-Wert abbildet. Die Double-Werte aller Elemente in der Liste sollen aufsummiert und zurückgegeben werden.
- (b) Verwenden Sie die Methode `LinkedList.sum` in den Klassen `Silo` und `Depot`, wo immer dies möglich und sinnvoll ist. Implementieren Sie die Funktion als Lambda-Ausdruck oder Methodenreferenz.

⁴Wikipedia: Palindromic number

⁵Wikipedia: Harshad number

Vergleichen Sie die alte und die neue Implementierung und diskutieren Sie die Vor- und Nachteile im Review. Im Falle von Methodenreferenzen sollten Sie in der Lage sein zu erklären, welche Art von Methodenreferenz Sie verwendet haben.

5. Aufgabe (Codeopolis)

- (a) Erweitern Sie die Klasse `Depot` um eine Methode `toString`, die ein `Predicate` und einen `Comparator` als Argumente erhält. Die Methode soll die Detailinformationen (einschließlich Informationen zu den `Harvest`-Objekten) der Silos, die dem `Predicate` entsprechen, in einen `String` schreiben und zurückgeben. Die Silos sollen mit dem `Comparator` sortiert werden.
- (b) Erweitern Sie den Dialog so, dass sich der Nutzer am Ende einer Spielrunde die Depotdetails anzeigen lassen kann. Der Benutzer soll die Anzeige der Silos filtern und sortieren können. Als Template können Sie die Klasse `DepotDetailsDialog` verwenden, die in Moodle zur Verfügung steht.